Tareas de la unidad 1 · Solucionario

Documento del profesorado · Corrección y rúbricas

Índice

1 Resumen **1**

2 T1.1 · Clasificar programas **2**
2.1 Respuestas típicas y correctas . . . 2
2.2 Dónde se equivocan casi siempre . . . 2
2.3 El apartado 2: estándar que parece a medida . . . 2
2.4 Rúbrica . . . 2

3 T1.2 · Quién traduce cada lenguaje **3**
3.1 Clave . . . 3
3.2 Los dos apartados de razonamiento . . . 3
3.3 Las dos trampas . . . 3

4 T1.3 · Fibonacci en tres lenguajes **4**
4.1 Qué tienen que encontrar . . . 4
4.2 Respuestas a las cinco preguntas . . . 4
4.3 Rúbrica . . . 4

5 T1.4 · Preguntas cortas · Clave **6**

1 Resumen

Tarea	Qué evalúa de verdad	Corrección
T1.1	Que sepan justificar una clasificación, no acertarla	Rúbrica, 10 puntos
T1.2	Que distingan traductor de lenguaje	Tabla con clave
T1.3	Que separen «el problema» de «el lenguaje»	Rúbrica, 10 puntos
T1.4	Teoría de la unidad	Clave de respuestas

Idea clave

Las tres primeras son de búsqueda, así que **la IA las resuelve en treinta segundos**. Por eso en las tres el peso está en la parte que la IA hace mal: justificar los casos dudosos, detectar las trampas y comparar con criterio propio.
Si quieres asegurarte, la defensa oral de dos minutos sobre lo entregado vale más que cualquier detector. Está previsto en la regla 2 del enunciado.

2 T1.1 · Clasificar programas

2.1 Respuestas típicas y correctas

Tipo	Lo que suelen poner bien
Sistema	Windows, Ubuntu, Android, controladores, GRUB, systemd, un antivirus, WinRAR (discutible), una máquina virtual
Programación	Eclipse, IntelliJ, PyCharm, Git, Maven, npm, Docker (discutible), Postman, DBeaver
Aplicación	Spotify, Blender, AutoCAD, GIMP, Telegram, Steam, Notion

2.2 Dónde se equivocan casi siempre

Programa	Lo que ponen	Lo defendible
WinRAR / 7-Zip	Aplicación	Discutible. Es una utilidad del sistema, pero el usuario la usa directamente. **Las dos valen si lo razonan**
Docker	Aplicación	Programación o sistema. Virtualiza entornos: se acerca más a sistema
Un antivirus	Aplicación	Sistema: se engancha al núcleo
Photoshop	Programación, «porque se crea con él»	Aplicación. Crear contenido no es programar
Android	Aplicación	Sistema: es un sistema operativo
Un navegador	Programación, por las DevTools	Aplicación. Llevar herramientas dentro no cambia lo que es

Ojo

El criterio de corrección es la justificación, no la etiqueta. Si alguien coloca Docker en «programación» y argumenta que lo usa el equipo de desarrollo para montar entornos reproducibles, es una respuesta de sobresaliente aunque otro libro lo ponga en «sistema».
Penaliza lo contrario: la tabla perfecta sin los dos párrafos de justificación.

2.3 El apartado 2: estándar que parece a medida

Respuestas buenas: **SAP**, **Odoo**, **Dynamics**, **WordPress**, **Moodle**, **Prestashop**.

La clave que tienen que decir: **configurar no es desarrollar**. El programa base es el mismo para todos los clientes, lo que cambia es la configuración y, como mucho, unos módulos añadidos. Si sale una actualización del fabricante, la reciben todos: eso no pasa con el software a medida.

2.4 Rúbrica

Criterio	Peso	Nivel alto	Nivel medio	Nivel bajo
Tabla completa	30 %	9 programas con las 6 columnas y fuentes oficiales	Faltan datos o fuentes	Menos de 9, o copiada de un listicle
Clasificación	20 %	Coherente y defendible	Algún error claro	Varios errores
Casos dudosos	30 %	Elige dos casos de verdad ambiguos y razona	Elige dos casos fáciles	No los hace
Estándar vs a medida	20 %	Distingue configurar de desarrollar	Confuso	Ausente

3 T1.2 · Quién traduce cada lenguaje

3.1 Clave

Lenguaje	Traductor más usado	Tipo	Qué genera
C	GCC	Compilador	Ejecutable nativo
C++	GCC (g++)	Compilador	Ejecutable nativo
Java	javac (del JDK)	Mixto	Fichero `.class` con bytecode
Python	CPython	Intérprete	Nada visible (cachea `.pyc`)
C#	Roslyn (del SDK de .NET)	Mixto	Ensamblado `.dll` con CIL
JavaScript	V8 (Chrome y Node)	Mixto, con JIT	Nada: traduce en memoria
Go	gc (el compilador oficial)	Compilador	Ejecutable nativo, sin dependencias

Alternativas que suelen encontrar, y están bien: Clang y MSVC para C y C++; OpenJDK frente al JDK de Oracle; PyPy, Jython o IronPython para Python; Mono para C#; SpiderMonkey y JavaScriptCore para JavaScript; TinyGo para Go.

Quién los mantiene: GCC → proyecto GNU (Free Software Foundation) · javac → Oracle y la comunidad OpenJDK · CPython → Python Software Foundation · Roslyn → Microsoft · V8 → Google · Go → Google. Todos son gratuitos.

3.2 Los dos apartados de razonamiento

1. Un lenguaje con dos traductores muy distintos. El caso más rico es **Python con CPython y PyPy**: el mismo lenguaje, pero PyPy usa compilación JIT y es mucho más rápido en programas largos, a costa de tardar más en arrancar y de no ser compatible con todo. Otro buen caso es **GCC frente a Clang**: mensajes de error mucho más claros en Clang, que fue justamente una de las razones de crearlo.

2. El cliente no debe ver el código fuente.

Sin problema	Con problema
C, C++, Go: entregas un binario	Python y JavaScript: entregas el fuente, es lo que se ejecuta
	Java y C#: entregas bytecode, que **no es el fuente pero se descompila con mucha facilidad**. Existen ofuscadores precisamente por eso

Idea clave

Esa última fila es la mejor respuesta posible y casi nadie la da sola. Si alguien la encuentra, que lo cuente en clase: enlaza directamente con por qué existen los ofuscadores y con la propiedad intelectual del software.

3.3 Las dos trampas

- **JavaScript.** Quien conteste solo «interpretado» tiene medio punto. La respuesta actual es que el motor **compila con JIT** mientras ejecuta: traduce a código máquina en memoria las partes que más se repiten. Nadie ejecuta JavaScript línea a línea desde hace más de quince años.
- **Java y C#.** Si alguien los pone como «compilados» sin más, pregúntale qué fichero sale y si ese fichero se puede ejecutar directamente. Sale solo.

4 T1.3 · Fibonacci en tres lenguajes

4.1 Qué tienen que encontrar

Lo único obligatorio es que los tres lenguajes sean de vías de traducción distintas. La combinación más frecuente será C + Python + Java, y está perfecta.

4.2 Respuestas a las cinco preguntas

1. Qué se parece. Lo que tienen que ver:

- Los **dos valores iniciales** (0 y 1, o 1 y 1) aparecen siempre.
- Hay una **suma de los dos anteriores** siempre.
- Hay un **bucle** o una **llamada a sí misma**, en los tres.
- Hace falta **guardar valores intermedios** en variables.

La conclusión que se busca: *el algoritmo es el mismo; lo que cambia es cómo se escribe*.

2. Qué cambia. Declaración de tipos (C y Java la exigen, Python no) · las llaves y los puntos y coma · la necesidad de una función `main` o una clase envolvente en C y Java · cómo se escribe en pantalla.

3. El más corto. Casi siempre Python. La respuesta buena **no** es «sí, el más corto es el más claro»: es que la brevedad ayuda **si ya conoces el lenguaje**, y que los tipos explícitos de Java hacen el código más largo pero también más fácil de revisar. Cualquiera de las dos posturas vale si está argumentada.

4. La pregunta que de verdad importa. Esta es la que conecta con toda la unidad:

Lenguaje	Qué hace falta instalar	Qué fichero se genera
C	Un compilador, solo para construirlo. Para ejecutarlo, nada	`.exe` o binario sin extensión
Python	El intérprete de Python, **siempre**	Ninguno visible (una caché `.pyc`)
Java	El JDK para compilar y el JRE para ejecutar	Un `.class` con bytecode

5. Recursivo. Que una función se llame a sí misma. Se da por buena cualquier formulación que capte esa idea. Si además alguien dice que la versión recursiva de Fibonacci es **muy lenta** porque repite los mismos cálculos, eso es nivel alto: es un tema de segundo curso.

4.3 Rúbrica

Criterio	Peso	Nivel alto	Nivel medio	Nivel bajo
Los tres lenguajes	15 %	De las tres vías distintas, bien elegidos	Dos de la misma vía	No cumple
Fuentes citadas	15 %	URL exacta y licencia comprobada	URL genérica	Sin fuentes
Parecidos y diferencias	25 %	Ve que el algoritmo es el mismo	Diferencias superficiales	Solo «uno es más corto»
Qué hace falta y qué se genera	35 %	Los tres casos correctos y razonados	Dos correctos	Confunde compilar con ejecutar
Recursividad	10 %	Explicada con sus palabras	Copiada	Ausente

Ojo

El apartado 4 es el 35 % a propósito: es el único que no se puede copiar de ningún sitio y el único que demuestra que han entendido la unidad. Si alguien entrega los tres códigos preciosos y el apartado 4 en blanco, no llega al 5.

5 T1.4 · Preguntas cortas · Clave

Software y tipos

1. Dos de estos cuatro: es lógico, no físico · se desarrolla, no se fabrica · no se estropea con el uso · una copia es idéntica al original.
2. Porque no sufre desgaste físico. Si falla a los diez años, el fallo ya estaba desde el principio: lo que ha cambiado es la entrada que ha recibido, no el programa.
3. El programa tiene una finalidad concreta; la aplicación es uno o varios programas que permiten al usuario hacer una tarea completa.
4. Abierta. Se valora que sean reales y que no repitan los de clase.
5. **Estándar.** Es un producto genérico configurado, y configurar no es desarrollar.

Código fuente y programa

6. Porque lo que se escribe es el código fuente; el programa ejecutable lo produce un traductor a partir de él.
7. El **código fuente**, porque es texto. El ejecutable es binario y sale ilegible.
8. **Las bibliotecas**: las funciones que el programador usa pero no ha escrito. Los huecos quedan sin rellenar.
9. El **enlazador**.
10. Código fuente → código objeto → ejecutable.

Los traductores

11. El compilador traduce todo el programa antes de ejecutarlo y produce un ejecutable. El intérprete lee, traduce y ejecuta línea a línea, y no produce nada.
12. Con compilador, **antes de ejecutar** y todos juntos. Con intérprete, **cuando el programa llega a esa línea**, lo que puede ser a los diez minutos de funcionamiento.
13. **Sí.** En un lenguaje interpretado, lo que se ejecuta es el propio código fuente, así que hay que entregarlo.
14. Un **código intermedio** (bytecode en Java, CIL en C#), que no es de ninguna máquina concreta.
15. Traduce ese código intermedio al código máquina del sistema donde se está ejecutando.
16. Porque no se traduce del todo al compilar: se traduce a un código intermedio universal, y cada sistema operativo tiene su propia máquina virtual que termina el trabajo. Lo portable es la máquina virtual, no el código.

Lenguajes

17. Lenguaje máquina → ensamblador → alto nivel.
18. Nivel · forma de traducción · propósito · paradigma.
19. Java: de **alto nivel** · **mixto** · de **propósito general** · principalmente **orientado a objetos** (hoy también con rasgos funcionales).
20. Que admite varias formas de organizar la solución a la vez. Python o Java, por ejemplo.

El proceso

21. Cuatro de: análisis de requisitos, diseño, codificación, pruebas, documentación, explotación, mantenimiento.
22. En el **mantenimiento**, alrededor del **80 %** del coste total.
23. Que da por supuesto que los requisitos están claros desde el principio y no van a cambiar. En la práctica el cliente siempre cambia de opinión, y en cascada volver atrás sale carísimo.
24. Dos de: editor (escribir) · compilador o intérprete (traducir) · enlazador (juntar con las bibliotecas) · depurador (parar y mirar dentro) · gestor de dependencias (descargar bibliotecas) · control de

versiones (histórico de cambios).

Idea clave

Puntuación sugerida: 24 preguntas, 0,4 puntos cada una sobre 10 (sobran 0,4 de propina). Las preguntas 5, 13, 16 y 23 son las que de verdad separan: si alguien acierta esas cuatro, ha entendido la unidad aunque falle detalles de otras.